GetItRight Production QA Plan

GetItRight Production QA Plan

1. Purpose and release decision

This document defines the quality-assurance process for the GetItRight polling, survey, verification, and administration platform.

Current release decision: HOLD.

The release is not production-ready until the backend integration fixture mismatch is resolved and the complete QA suite passes. The current codebase does pass Python syntax validation, .NET Release compilation, and Docker Compose configuration validation.

2. System under test

Component	Location	Technology	QA scope
Python API	backend/api/	FastAPI, SQLAlchemy, SQLite	API, auth, polls, surveys, WebSockets, middleware
Enterprise API	backend/csharp/GetItRight.Api/	.NET 8	Build, API compatibility, persistence
Mobile client	mobile/	Flutter/Dart	Static analysis, unit/widget tests, release build
Web client	web/	Static HTML/CSS/JavaScript, Nginx	Browser smoke tests, accessibility, verifier flows
Native modules	native/	C/C++	Compiler and ABI validation where integrated
Event schema	shared/proto/	Protocol Buffers	Schema compatibility
Rust engine	engine/	Rust	Not currently buildable: no Cargo manifest is present
Deployment stack	docker-compose.yml	Docker Compose	Configuration, image build, startup, health checks

3. Environment requirements

Required for the full release gate:

- Python 3.12 or compatible Python 3.10+

- The project virtual environment at backend/.venv
- .NET SDK 8
- Flutter SDK 3.x or later
- Docker Engine and Docker Compose
- A browser capable of running the web smoke tests

The repository currently has Python, .NET, Rust, Docker, and Node/npm available. Flutter is not installed in the current environment. There is no package.json and no Rust Cargo.toml, so no Node package installation or Rust workspace build can currently be performed.

4. Installation and setup

Python

```
cd /path/to/POLL
python3 -m venv backend/.venv
backend/.venv/bin/python -m ensurepip --upgrade
backend/.venv/bin/python -m pip install -r backend/requirements.txt
backend/.venv/bin/python -m pip install pytest
```

.NET

```
dotnet restore backend/csharp/GetItRight.Api/GetItRight.Api.csproj
```

Flutter

Install the Flutter SDK using the official Flutter installation procedure, then run:

```
cd mobile
flutter pub get
```

Do not commit SDKs, generated build output, credentials, or local database files to the repository.

Environment configuration

Create a local .env file from .env.example and provide:

- SECRET_KEY: a unique random value of at least 32 characters
- CORS_ORIGINS: explicit trusted HTTPS origins for production
- ENVIRONMENT=production

Never place a real secret in test code, documentation, shell history, or source control. For local-only validation, a temporary process environment variable may be used:

```
SECRET_KEY='replace-with-a-local-random-secret-of-32-or-more-characters'
```

5. QA test matrix

Gate A: source and static validation

```
backend/.venv/bin/python -m compileall -q backend/api backend/tests
dotnet build backend/csharp/GetItRight.Api/GetItRight.Api.csproj \
  --configuration Release
cd mobile && flutter analyze
```

Expected result: no syntax errors, no build errors, and no analyzer errors.

Gate B: backend integration tests

```
SECRET_KEY='local-test-secret-at-least-32-characters-long' \
  backend/.venv/bin/python -m pytest -q backend/tests
```

The suite in test_all_endpoints.py covers:

1. Health checks
2. Administrator authentication
3. Seed-user authentication
4. User registration
5. Login for a newly registered user
6. Plan changes
7. Administrator status changes
8. Poll listing
9. Vote submission and receipt generation
10. Survey listing
11. Survey submission
12. Checkpoint retrieval
13. Checkpoint publication

Gate C: deployment configuration

```
SECRET_KEY='local-test-secret-at-least-32-characters-long' \
  docker compose config
```

Then, in an environment with the required secret configured:

```
docker compose build --pull
docker compose up -d
curl --fail http://127.0.0.1:5000/health
curl --fail http://127.0.0.1:8888/
docker compose ps
docker compose logs --no-log-prefix --tail=200 backend web
docker compose down
```

The deployment gate requires both health endpoints to respond successfully, containers to remain running, and logs to contain no startup exceptions, credential errors, database errors, or repeated restart loops.

Gate D: web smoke testing

Run against the deployed web service:

- Landing page loads without console errors.
- Login and signup validation reject malformed input.
- Poll list loads and status filters work.
- A vote can be submitted once and duplicate voting is rejected.
- Survey list and survey submission work.
- Evidence verifier computes and displays a SHA-256 result.
- Currency selection updates visible pricing.
- Keyboard navigation and focus indicators work on primary flows.
- Responsive layouts work at mobile, tablet, and desktop widths.

Gate E: mobile validation

```
cd mobile
flutter analyze
```

```
flutter test
flutter build apk --release
```

The release APK must install on a supported Android device or emulator and must pass login, poll browsing, voting, survey submission, profile/settings, logout, and offline/poor-network smoke checks.

6. Current evidence

Executed in the current workspace:

Check	Result	Evidence
Python dependency installation	PASS	All pinned requirements already satisfied after bootstrapping venv pip
Python syntax compilation	PASS	`compileall` completed successfully
Backend integration suite	FAIL	Admin login fixture fails with HTTP 401
.NET dependency restore	PASS	Project up to date
.NET Release build	PASS	0 warnings, 0 errors
Docker Compose config	PASS	Valid with process-only test secret
Flutter analysis/tests/build	BLOCKED	Flutter SDK is not installed
Rust engine build	BLOCKED	No `Cargo.toml` or `Cargo.lock` exists
Node/web package install	NOT APPLICABLE	No `package.json` exists

7. Blocking defect

The backend integration test and Python seed data do not describe the same accounts:

- The test expects `admin@getitright.io` with `SuperAdminPassword2026!`.
- The Python seed in seed.py creates users such as `grace@getitright.io` with passwords `pass1` through `pass15`.
- The test also expects `grace@umoja.org`, which is not created by the Python seed.

Resolve this by choosing one canonical fixture source and updating both the seed and test data to match. Production credentials must not be used as test fixtures. Prefer dedicated test-only accounts and an isolated test database. After the fix, rerun Gate B and confirm all 13 flows pass.

8. Production release gates

Release approval requires all of the following:

- Backend integration suite passes with an isolated test database.
- No hard-coded production credentials remain in tests or seed scripts.
- Python syntax and dependency checks pass.

- .NET Release build passes with zero warnings and errors.
- Flutter analyze, tests, and release build pass.
- Docker images build from a clean checkout.
- Containers start with production environment variables only.
- Health checks, logs, and restart behavior pass.
- CORS contains only approved HTTPS origins.
- Security headers, authentication, authorization, rate limiting, and token expiry are verified.
- Web and mobile smoke tests pass on supported browsers/devices.
- Backup, restore, migration, and rollback procedures are tested.
- No critical or high security findings remain open.
- QA evidence is attached to the release record.

9. Recommended CI pipeline

CI should execute these stages in order:

1. Dependency restore with lockfile/pinned-version verification.
2. Python syntax and unit/integration tests.
3. .NET restore and Release build/test.
4. Flutter dependency restore, analyzer, tests, and release build.
5. Docker Compose configuration and image build.
6. Container startup and health smoke tests.
7. Dependency and image vulnerability scanning.
8. Artifact retention for test reports, logs, and build outputs.

CI must inject secrets through the CI secret store. It must never generate, commit, or print `.env` contents or tokens in logs.